

PCD T17 – Semantic Analyzer for Source Code

Echipa proiect PCD

2026

1 Descriere generala

Acest proiect este realizat pentru materia Programare Concurenta si Distribuita si are ca tema T17 – Analiza Semantica Cod Sursa.

Aplicatia implementeaza un sistem client-server in C, care permite trimiterea de fisiere sursa C catre un server. Serverul salveaza fisierele primite, porneste procesul de analiza semantica si genereaza rapoarte text cu rezultatele obtinute.

Analiza codului este realizata folosind biblioteca `libclang`. Aceasta permite parsarea fisierelelor C si extragerea unor informatii precum:

- numarul de diagnostice;
- erori si warning-uri;
- numarul de functii;
- numarul de variabile;
- numarul de instructiuni `if`;
- numarul de bucle `for`;
- numarul de bucle `while`.

Proiectul include si un client de administrare, un client normal in C si un client alternativ in Python.

2 Componentele proiectului

Proiectul este impartit in mai multe componente:

- `server` – server TCP concurent;
- `client` – client normal in C;
- `admin_client` – client de administrare;
- `analyzer` – modul de analiza semantica bazat pe `libclang`;
- `python_client/client.py` – client alternativ in Python;

- `tests/` – fișiere sursa folosite pentru testare;
- `config/` – fișiere de configurare;
- `docs/` – documentatia proiectului.

3 Structura proiectului

```
.
+-- Makefile
+-- README.tex
+-- config
| +-- app.cfg
| '-- users.cfg
+-- docs
| +-- SRS_SDD.tex
| +-- openapi.yaml
| '-- protocol.tex
+-- python_client
| '-- client.py
+-- src
| +-- admin_client.c
| +-- analyzer.c
| +-- client.c
| +-- main.c
| '-- server.c
+-- tests
| +-- bad.c
| +-- bad_syntax.c
| +-- bad_type.c
| +-- bad_uninitialized.c
| +-- good_math.c
| +-- good_nested.c
| +-- good_while.c
| '-- sample.c
+-- uploads
+-- reports
+-- downloads
+-- downloads_py
'-- logs
```

Folderele `uploads`, `reports`, `downloads`, `downloads_py` si `logs` sunt folosite pentru fișiere generate in timpul rularii aplicatiei.

4 Cerinte software

Pentru compilare si rulare sunt necesare:

- sistem Linux sau WSL;

- gcc;
- make;
- libclang;
- libconfig;
- python3, pentru clientul alternativ.

Pe Ubuntu/WSL, pachetele pot fi instalate cu:

```
sudo apt update
sudo apt install build-essential make clang libclang-dev libconfig-dev python3
```

În implementarea curentă, Makefile-ul folosește LLVM 18:

```
-I/usr/lib/llvm-18/include
-L/usr/lib/llvm-18/lib
-lclang
-lconfig
```

Dacă versiunea de LLVM diferă, trebuie ajustate căile din Makefile.

5 Compilare

Pentru compilarea întregului proiect:

```
make clean
make
```

Comanda `make` generează următoarele executabile:

- `server`;
- `client`;
- `admin_client`;
- `analyzer`;
- `main`.

Compilarea se face cu opțiuni stricte:

```
-Wall -Wextra -Werror -pedantic -std=c11
```

6 Rulare server

Serverul se pornește cu:

```
./server
```

Serverul rulează pe adresa:

```
127.0.0.1:8080
```

Serverul primește conexiuni de la clienți, autentifică utilizatorii, salvează fișierele în `uploads/`, rulează analiză și generează rapoarte în `reports/`.

7 Rulare client normal

Clientul normal poate trimite un fisier sursa catre server:

```
./client tests/sample.c
```

Exemplu pentru un fisier cu eroare:

```
./client tests/bad_type.c
```

Rezultatul analizei este afisat in terminal si salvat ca raport in folderul **reports/**.

8 Rulare client Python

Clientul Python ofera o varianta alternativa de client, intr-un alt limbaj.

Pentru upload:

```
python3 python_client/client.py upload tests/sample.c
```

Pentru download raport:

```
python3 python_client/client.py download sample_report.txt
```

Fisierele descarcate de clientul Python sunt salvate in **downloads_py/**.

9 Rulare client admin

Clientul de administrare se porneste cu:

```
./admin_client
```

Meniul permite urmatoarele actiuni:

- afisarea statisticilor serverului;
- afisarea logurilor;
- listarea fisierelor uploadate;
- listarea rapoartelor generate;
- inchiderea clientului.

Exemplu de meniu:

```
=== Admin Client ===  
  
1. Server stats  
2. Show server logs  
3. List uploaded files  
4. List reports  
5. Quit
```

10 Fișiere de test

Folderul `tests/` conține fișiere folosite pentru demonstrarea funcționalităților:

Fisier	Descriere
<code>sample.c</code>	fișier simplu, fara erori
<code>bad.c</code>	fișier cu problema semantica
<code>bad_syntax.c</code>	fișier cu eroare detectata la parsare
<code>bad_type.c</code>	fișier cu eroare de tip
<code>bad_uninitialized.c</code>	fișier cu variabila neinitializata
<code>good_math.c</code>	fișier valid cu functii si bucle
<code>good_nested.c</code>	fișier valid cu structuri <code>if</code> imbricate
<code>good_while.c</code>	fișier valid cu bucla <code>while</code>

11 Exemplu de rezultat

Pentru rularea:

```
./client tests/sample.c
```

se poate obtine un rezultat de forma:

```
=== Analysis Result ===
Project: PCD T17 Semantic Analyzer
Source file: uploads/sample.c
Diagnostics count: 0
Functions count: 3
Variables count: 8
If statements count: 1
For loops count: 1
While loops count: 0

=== Diagnostics ===
Nu exista diagnostice.

Parse completed.
```

12 Test de concurenta

Serverul poate procesa mai multi clienti. Un test simplu se poate face astfel:

```
./client tests/sample.c & ./client tests/bad_type.c & wait
```

Dupa rulare, rapoartele trebuie sa apara in folderul `reports/`.

13 Curatare proiect

Pentru curatarea executabilelor:

```
make clean
```

Folderul `uploads/` poate fi sters automat de Makefile. Folderele generate sunt ignorate prin `.gitignore`, dar sunt pastrate in repository prin fisiere `.gitkeep`.

14 Scenariu recomandat pentru demo

Un demo complet poate fi facut cu urmatoorii pasi:

```
make clean
make
./server
```

Intr-un alt terminal:

```
./client tests/sample.c
./client tests/bad_type.c
python3 python_client/client.py upload tests/good_while.c
python3 python_client/client.py download good_while_report.txt
./admin_client
```

Acest scenariu demonstreaza:

- compilarea fara warning-uri;
- server TCP functional;
- client C functional;
- client Python functional;
- analiza semantica prin `libclang`;
- generare de rapoarte;
- client admin;
- loguri si statistici.

15 Concluzie

Proiectul implementeaza o aplicatie distribuita simpla, dar completa, pentru analiza semantica de cod sursa. Aplicatia foloseste socket-uri TCP, procese, fisiere de configurare, loguri, client normal, client admin si client Python. Partea de analiza este realizata cu `libclang`, iar rezultatele sunt salvate si transmise clientului sub forma de rapoarte text.